A MICRO PROJECT REPORT

ON

"Cryptarithmetic Puzzle Solver using CSP Backtracking"

Subject: Foundation of Artificial Intelligence
(Course Code: 2ADPC204)

Submitted in partial fulfilment of the requirement for
the award of the Degree in SY (AI & DS).

SUBMITTED BY
Shreya Umesh Gadale | Roll No. 2050

UNDER THE GUIDANCE OF
Prof. Ms. Priyanka C Patil

Annasaheb Dange College of Engineering and Technology, Ashta
Department of AI & DS

Academic Year: 2026 – 2027

CERTIFICATE

This is to certify that Ms. Shreya Umesh Gadale of SY B. Tech (AI & DS), Roll No. 2050, of Annasaheb Dange College of Engineering and Technology, Ashta, has successfully completed the micro project entitled ***"Cryptarithmetic Puzzle Solver using CSP Backtracking"*** in the subject Foundation of Artificial Intelligence (Course Code: 2ADPC204) for the academic year 2026–2027 as prescribed in the curriculum.

Place: _____

Date: _____

Subject Teacher

(Signature)

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my guide, Prof. Ms. Priyanka C patil, for valuable guidance and support throughout this micro project on “Cryptarithmetic Puzzle Solver using CSP Backtracking.” I am thankful to the Head of the Department Dr Asma Shaikh, the Director Mr L.Y.Waghmode, and the faculty of Annasaheb Dange College of Engineering and Technology, Ashta, for providing the facilities and encouragement required to complete this work. Finally, I thank my family and friends who helped me directly or indirectly during the project.

Shreya Umesh Gadale

Roll No.: 2050

INDEX

Sr. No.	*Title*	*Page No.*
1	Aims / Benefits of the Micro Project	5
2	Course Outcomes Achieved	5
3	Literature Review	5
4	Actual Methodology Followed	6
5	Actual Resources Used	6
6	Algorithm and CSP Backtracking	7
7	Program / Implementation	8
8	Outputs of the Micro Project	9–10
9	Skills Developed and Applications	11
10	Conclusion and References	11–12

MICRO PROJECT REPORT

CRYPTARITHMETIC PUZZLE SOLVER USING CSP BACKTRACKING

1. Aims / Benefits of the Micro Project

Aim: To develop a cryptarithmetic puzzle solver that treats letters as variables and digits as values, then uses Constraint Satisfaction Problem (CSP) techniques and backtracking to find a valid digit assignment for an arithmetic puzzle such as SEND + MORE = MONEY.

Benefits:

- Solvesletter-based arithmetic puzzles systematically.
- Demonstrates CSP variables, domains, and constraints.
- Uses backtracking to undo incorrect assignments.
- Column pruning checks arithmetic constraints early and avoids unnecessary search.
- Improves logical reasoning and problem-solving skills.

2. Course Outcomes Achieved

1. Understood the basic concepts of Artificial Intelligence and constraint satisfaction.
2. Identified variables, digit domains, and constraints in a puzzle.
3. Applied recursion and backtracking to search for a solution.
4. Used uniqueness and leading-digit constraints to reject invalid assignments.
5. Improved logical thinking, testing, and debugging skills.

3. Literature Review

Cryptarithms are mathematical puzzles in which letters represent digits. A valid solution assigns one digit to each distinct letter, does not reuse a digit for another letter, and makes the arithmetic equation correct. A CSP represents a problem through variables, domains, and constraints. Backtracking builds an assignment step by step and reverses a choice when it violates a constraint. Column-wise checking (pruning) can reject invalid partial assignments before every letter is assigned, making the search more efficient.

4. Actual Methodology Followed

1. Selected the puzzle SEND + MORE = MONEY.
2. Listed the unique letters: S, E, N, D, M, O, R and Y.
3. Assigned each letter a digit from 0 to 9.
4. Added the all-different constraint so no two letters share a digit.
5. Prevented leading letters S and M from being zero.
6. Used backtracking to assign digits and undo choices when constraints failed. 7. Applied column-wise arithmetic checks to prune invalid partial assignments. 8. Converted the final letter assignments into numbers and verified the sum.

5. Actual Resources Used

Hardware: Computer or laptop, keyboard and display.
Software: Windows / Android browser for viewing the interface; Python 3; Visual Studio Code or another code editor; HTML, CSS and JavaScript for the website interface.
Other resources: Puzzle examples, Python documentation, class notes, and testing inputs.

Problem Definition

SEND + MORE = MONEY

Each distinct letter represents a different digit. The first letters of the numbers (S and M) cannot be zero. The solver searches for assignments that satisfy the arithmetic equation and all digit constraints.

CSP Components

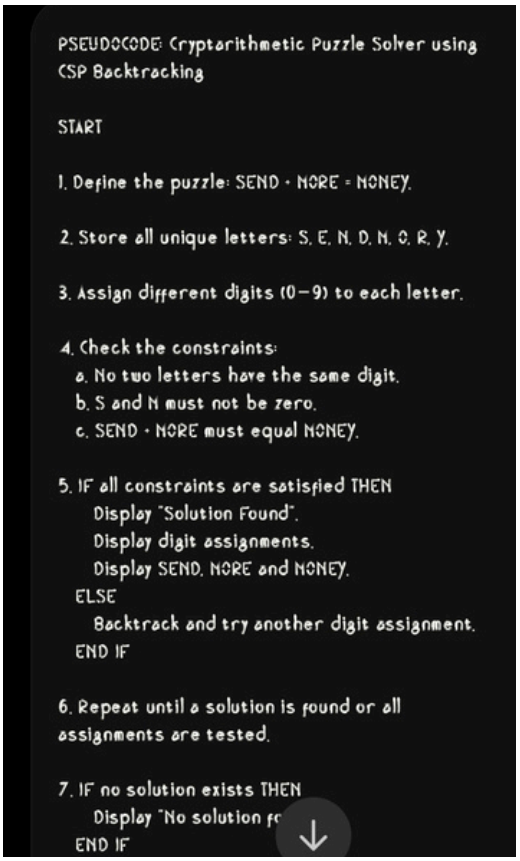
Variables: S, E, N, D, M, O, R, Y.
Domains: Digits 0–9 (with S and M restricted to 1–9).
Constraints: All letters have different digits; the equation must be true; leading digits cannot be zero.

6. Algorithm and CSP Backtracking

Algorithm:

1. Start with an empty assignment.
2. Select an unassigned letter.
3. Try an unused digit from its domain.
4. Reject zero for a leading letter.
5. Check any arithmetic columns that are already determined.
6. If the partial assignment is valid, recursively assign the next letter.
7. If no digit works, remove the last assignment and try another digit (backtrack). 8. When all letters are assigned, verify SEND + MORE = MONEY.
9. Display the mapping and numerical equation.

Pseudocode



```
PSEUDOCODE: Cryptarithmetic Puzzle Solver using
CSP Backtracking

START

1. Define the puzzle: SEND + MORE = MONEY.

2. Store all unique letters: S, E, N, D, M, O, R, Y.

3. Assign different digits (0–9) to each letter.

4. Check the constraints:
  a. No two letters have the same digit.
  b. S and M must not be zero.
  c. SEND + MORE must equal MONEY.

5. IF all constraints are satisfied THEN
     Display "Solution Found".
     Display digit assignments.
     Display SEND, MORE and MONEY.
  ELSE
     Backtrack and try another digit assignment.
  END IF

6. Repeat until a solution is found or all
assignments are tested.

7. IF no solution exists THEN
     Display "No solution fo
  END IF
```

Column Pruning

Column pruning checks units, tens, hundreds, and higher columns as soon as enough digits are known. If a column cannot satisfy its sum and carry, that partial assignment is abandoned immediately. This reduces the number of combinations explored compared with plain backtracking.

Expected Result

A valid assignment is found for SEND + MORE = MONEY. The output screenshot on the next page shows the digit mapping and verifies the equation.

7. Program / Implementation

The core solver can be implemented in Python using recursion and backtracking. The website screenshot included in this report shows the browser interface with puzzle choices and solver options.

Core Python Logic (illustrative)

```
from itertools import permutations

def solve_cryptarithmetic():
    letters = 'SENDMORY'

    # Try different digit assignments
    for digits in permutations(range(10),
len(letters)):
        d = dict(zip(letters, digits))

        # First letters cannot be zero
        if d['S'] == 0 or d['M'] == 0:
            continue

        # Convert words into numbers
        SEND = (d['S'] * 1000 + d['E'] * 100
                + d['N'] * 10 + d['D'])

        MORE = (d['M'] * 1000 + d['O'] * 100
                + d['R'] * 10 + d['E'])

        MONEY = (d['M'] * 10000 + d['O'] * 1000
                 + d['N'] * 100 + d['E'] * 10
                 + d['Y'])

        # Check the equation
        if SEND + MORE == MONEY:
            print("Solution Found!")
            print("--------------------")

            for letter in letters:
                print(f"{letter} = {d[letter]}")

            print("--------------------")
            print("SEND =", SEND)
            print("MORE =", MORE)
            print("MONEY =", MONEY)
            print("--------------------")
            print(f"{SEND} + {MORE} = {MONEY}")
            return

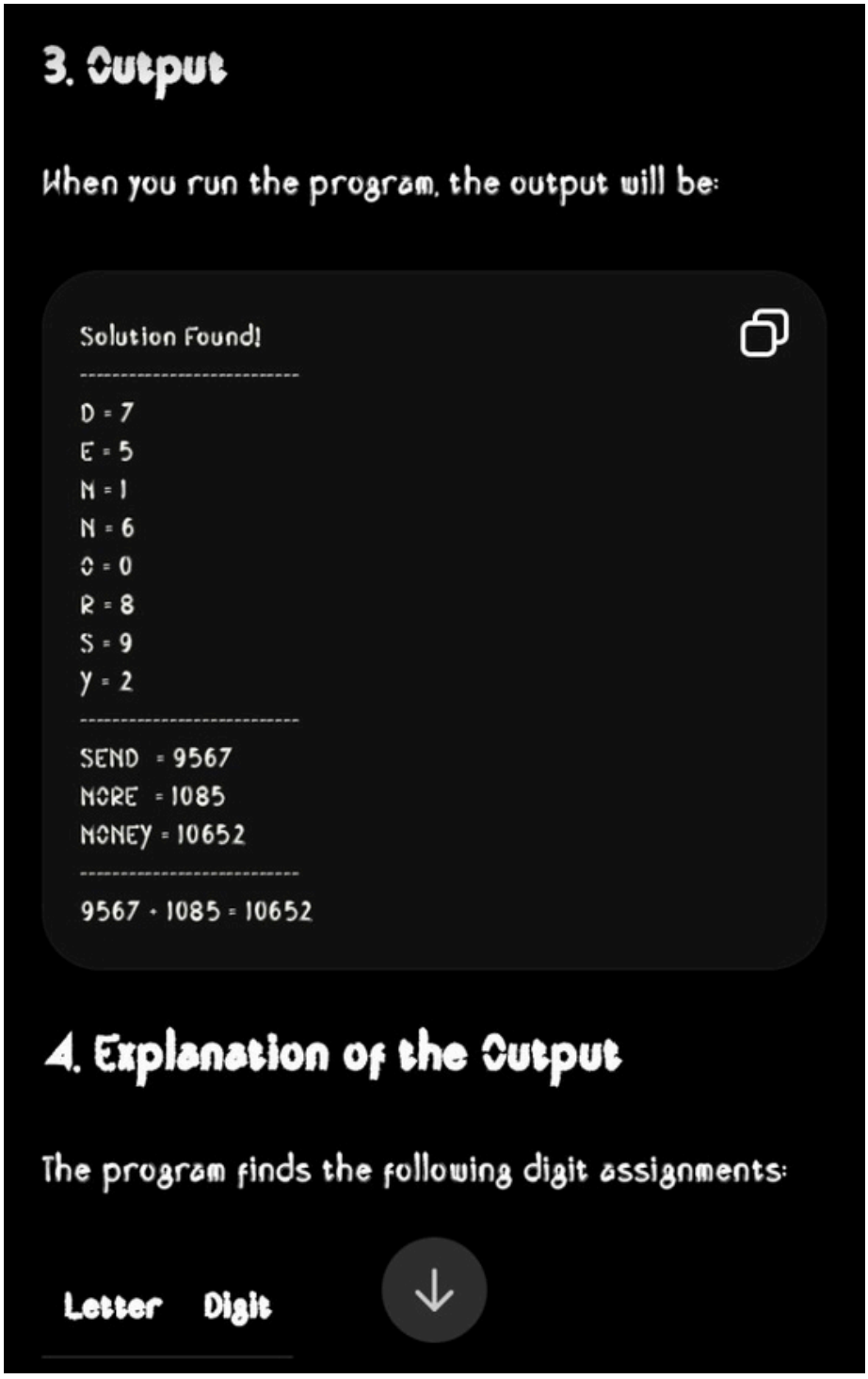
    print("No solution found.")
```

Main Features

- Multiple puzzle presets shown in the website interface.
- Solve button to run the selected puzzle.
- Column pruning and plain backtracking options.
- Option to count all solutions.
- Displays letter-to-digit assignments and the verified arithmetic equation.

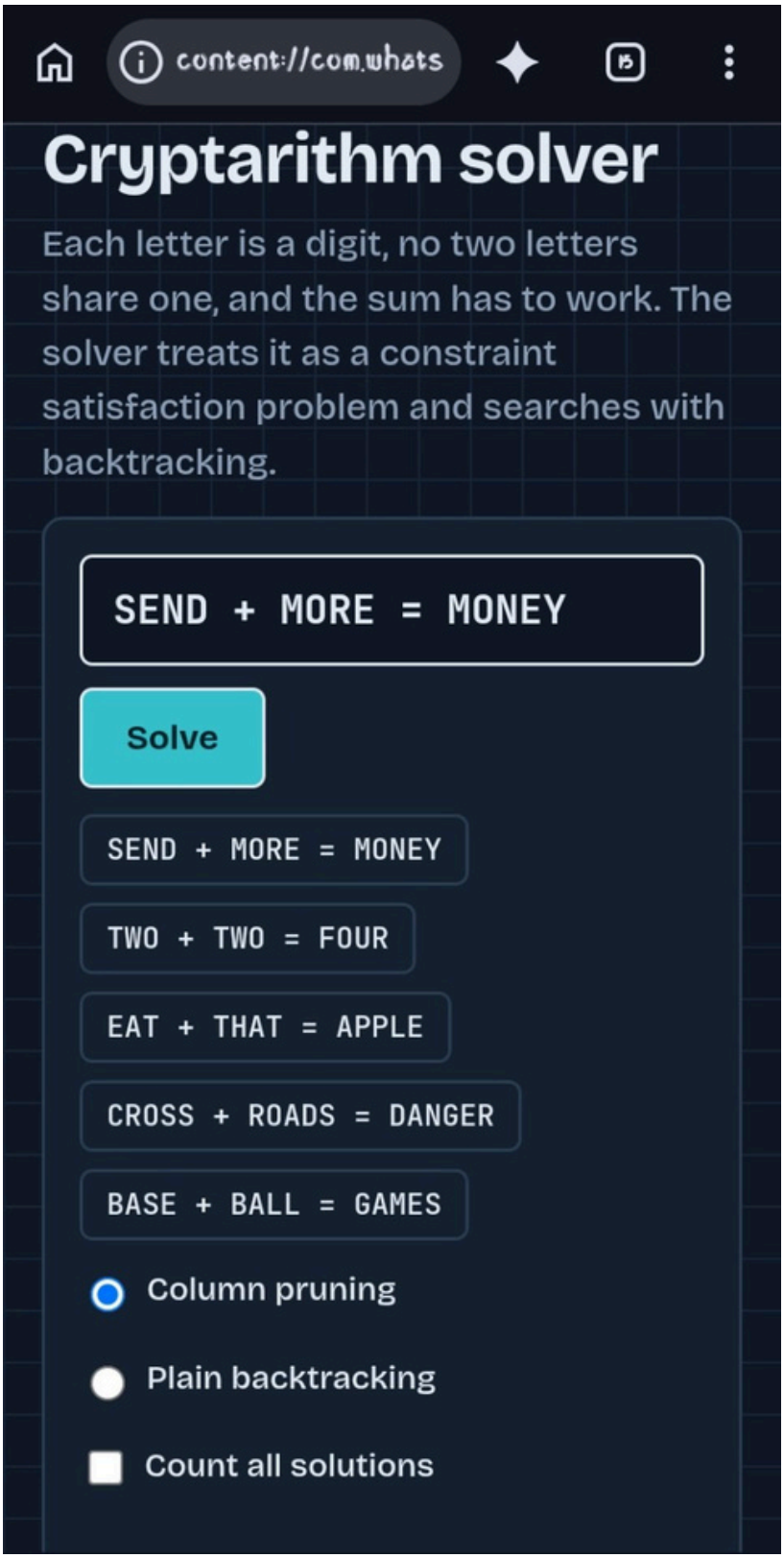
8. Outputs of the Micro Project — Solver Output

The output below is the supplied result screenshot for SEND + MORE = MONEY. It shows a valid digit mapping and confirms that the addition is correct.



Website Interface Screenshot

The following screenshot shows the Cryptarithmetic Solver web interface, the selected puzzle, available puzzle examples, and CSP solving options.



9. Skills Developed

• Python programming and logical reasoning.
• Understanding of CSP variables, domains, and constraints.
• Recursive problem-solving and backtracking.
• Constraint checking and search-space pruning.
• Testing, debugging, and explaining program output.
• Basic web interface understanding.

10. Applications of the Micro Project

1. Educational tools for teaching arithmetic and AI search.
2. Puzzle-solving websites and applications.
3. Demonstrations of constraint satisfaction problems.
4. Introduction to search algorithms and recursive programming.
5. AI coursework, practical demonstrations, and logic exercises.

Conclusion

The Cryptarithmetic Puzzle Solver demonstrates how a mathematical word puzzle can be modelled as a Constraint Satisfaction Problem. Each letter is assigned a unique digit, leading digits are kept non-zero, and the arithmetic equation is checked. Backtracking explores possible assignments while column pruning rejects invalid partial solutions early. The supplied output verifies SEND = 9567, MORE = 1085, and MONEY = 10652, so 9567 + 1085 = 10652.

References

1. Python Documentation: https://docs.python.org/3/
2. Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach, Pearson.
3. Class notes on Constraint Satisfaction Problems and Backtracking.
4. User-provided website and output screenshots.